JewlX Platform

Cron Jobs & Database Load Report

Tamam scheduled jobs ki inventory, timing, processing load, cost aur DB-pressure ka hal

Item	Detail
Report date	26 October 2026
Project	Supabase project xyrmsozhjtqxvrviykjf (JewlX) - Large instance (8 GB) per Lovable
Source	Repo ki supabase/migrations/*.sql, supabase/functions/*, git history, aur Lovable ka 24-Oct database-pressure response
Method	Static code analysis. Live DB se run-durations nahi parhi gayi (is PC se Supabase CLI/MCP connect nahi). Isliye durations 'estimate' hain; Appendix A mein woh SQL hai jis se aap exact numbers nikaal sakte hain.
Timezone	Sab waqt UTC mein; bracket mein PKT (UTC+5). Pakistan mein DST nahi hota.

1. Khulasa (Executive Summary)

Repo ke mutabiq is waqt **14 pg_cron jobs schedule** hain, aur **1 job sirf live DB mein** hai (repo se revert ho chuki, lekin DB mein chal rahi hai), yani **total 15 active cron jobs**. Is ke ilawa 2 purani jobs unschedule ho chuki hain aur 1 dynamic job (marketing tick) sirf campaign ke doran banti hai.

Roz ke hisaab se **takreeban 1,740 cron executions** hoti hain, lekin 99% halki hain (1 minute wala diamond tick aksar 'idle' return karta hai). **Asal DB load sirf 3 jobs se aata hai:**

- **Diamond pipeline (tick + nightly 03:05 UTC):** poore 794k-row diamond pool ka full scan, scoring, 794k rows ka TRUNCATE+INSERT, 3 materialized views ka refresh aur 9 DISTINCT scans - ek run mein. Statement timeout 1 ghanta rakha gaya hai. **Sab se bhari job.**
- **daily-jewlx-db-backup (20:00 UTC = 01:00 PKT):** edge function HAR public table ko 1000-row pages mein OFFSET pagination se poora download karti hai - diamond_inventory_pool (794k) aur diamond_pool_scores (794k) samet. OFFSET pagination bade tables par O(n^2) ban jati hai. 24 Oct 22:02 UTC ka alert backup shuru hone ke theek 2 ghante baad aaya jab pool 2.15M rows tha. **Lovable ne is job ko bilkul miss kiya.**
- **refresh_stale_diamond_availability_5min (har 5 min):** har confirmed client ki availability 24 ghante baad dobara compute hoti hai; har compute = pool ka 1-2 full scan with per-row plpgsql function (index use nahi hota). Ye job repo mein nahi, sirf live DB mein hai.

Verdict on Lovable: Lovable ka ye kehna sahi hai ke abhi instance upgrade karne ki zaroorat nahi aur diamond nightly job bhari hai. Lekin uska analysis adhoora hai: backup job aur 5-minute availability sweep (dono full-pool scans) uski list mein nahi. Agar sirf 'diamond jobs cap' kiye gaye to raat 20:00 UTC wala spike phir bhi aayega.

Hal ek line mein: instance barhane ki bajaye 3 bhari jobs ko theek karein (Section 6). Us ke baad DB itni halki ho jaye gi ke **Medium (4 GB) par wapas jaana safe** hoga, jo Lovable ke mutabiq ~210 credits/month bachata hai.

2. Cron Jobs ki Mukammal Inventory

Load rating: **HIGH** = full-table scans / lakhon rows; **MED** = row-by-row loops ya bade DELETE; **LOW** = chand rows ya sirf HTTP call. Duration estimate 794k-row pool par Large instance ke liye hai; exact numbers Appendix A ki SQL se nikalein.

#	Job name (cron.job)	Schedule (UTC) / PKT	Runs/ day	Kya karti hai	Load	Est. duration	Repo file
1	diamond-pipeline-tick	* * * * * (har minute)	1,440	Agar pool 'pending' ho (upload ya nightly flag) aur 2-min debounce guzar chuka ho to score_diamond_pool() + 3 matview REFRESH + distinct-values cache. Warna 1 row parh kar 'idle' return. statement_timeout = 1 ghanta.	HIGH (jab chale)	idle: ~2 ms. Full run: 10-60 min	20260925090000, 20261025120000
2	diamond-pipeline-nightly	5 3 * * * (03:05 / 08:05 PKT)	1	Sirf pending=true set karta hai -> #1 ko 03:07 par full run karwata hai, chahe pool badla ho ya nahi.	HIGH (indirect)	flag: 1 ms; triggered run: 10-60 min	20260925090000
3	refresh_stale_diamond_availability_5min	*/5 * * * *	288	5 confirmed clients jin ka cache stale ho YA 24h purana ho: har ek ke liye compute _client_diamond_catalog_availability() = pool full scan with diamond_pool_row_matches() per row (plpgsql, index-less). LIVE DB ONLY - repo se revert ho chuki.	HIGH (per client scan)	empty: ~5 ms. Per client: 5-40 s; 5 clients: upto 3 min	deleted file 20260910130515 (git ff79772d)
4	daily-jewlx-db-backup	0 20 * * * (20:00 / 01:00 PKT)	1	net.http_post -> database-backup edge fn: list_public_tables() -> HAR table select('*') 1000-row OFFSET pages -> JSON gzip -> Storage. diamond_inventory_pool + diamond_pool_scores (~1.6M rows) bhi. Anon JWT cron command mein hard-coded.	HIGH	30 min - 2+ ghante (OFFSET O(n^2)); edge fn memory/time limit se partially fail hone ka imkaan	20260421135203
5	cleanup_orphan_ai_previews_daily	0 3 * * * (03:00 / 08:00 PKT)	1	storage.objects mein 60 din se purane AI previews par FOR loop; har object ke liye 4 tables mein EXISTS check, phir DELETE. #2 se sirf 5 min pehle.	MED	1-10 min (objects ki tadaad par)	20260422182534
6	nightly-stripe-reconciliation	0 2 * * * (02:00 / 07:00 PKT)	1	http_post -> admin-reconciliation: Stripe subscriptions list + per-sub retrieve, client_subscriptions upsert. DB par halka, Stripe API par bhari.	LOW	30 s - 3 min (Stripe API)	20260418160000
7	expire-purchased-credits	5 0 * * * (00:05 / 05:05 PKT)	1	client_wallets loop: 365 din purane balance zero + wallet_transactions insert.	LOW	< 1 s	20260416165255
8	daily-leads-report	0 0 * * * (00:00 / 05:00 PKT)	1	http_post -> daily-leads-report edge fn -> get_daily_leads_report RPC (24h window) -> email.	LOW	2-10 s	20260921180000
9	platform-health-daily-rollup	15 0 * * * (00:15 / 05:15 PKT)	1	rollup_platform_health_daily(yesterday) + prune 400-din purana health data.	LOW	< 2 s	20260918180000
10	platform-health-monitor	0 6 * * * (06:00 / 11:00 PKT)	1	http_post -> platform-health-monitor edge fn: service pings, cron.job_run_details reads, quota snapshots, incidents, alerts email.	LOW-MED	10-40 s	20260918180000, 20260929120000
11	grant-monthly-free-credits	0 0 1 * * (1st, 00:00)	1/month	client_free_credits delete + insert for all clients.	LOW	< 2 s	20260417100000
12	trim_old_log_tables_monthly	0 4 1 * * (1st, 04:00)	1/month	Ek hi statement mein DELETE analytics_events + client_activity_logs WHERE created_at < 90 days. analytics_events high-volume hai (Lovable: page visits sab se frequent query) -> mahine mein ek bara DELETE = WAL/IO spike + table bloat.	MED-HIGH (1x/ month)	1-15 min (rows par)	20260422182534
13	archive_old_ai_usage_logs_monthly	0 4 2 * * (2nd, 04:00)	1/month	ai_usage_logs 90 din se purane -> monthly summary upsert -> DELETE.	MED (1x /month)	10 s - 3 min	20260319121415
14	platform-health-monthly-report	0 8 1 * * (1st, 08:00)	1/month	Pichle mahine ke exact counts (count: exact) multiple tables par + email report.	LOW-MED	10-60 s	20260918180000

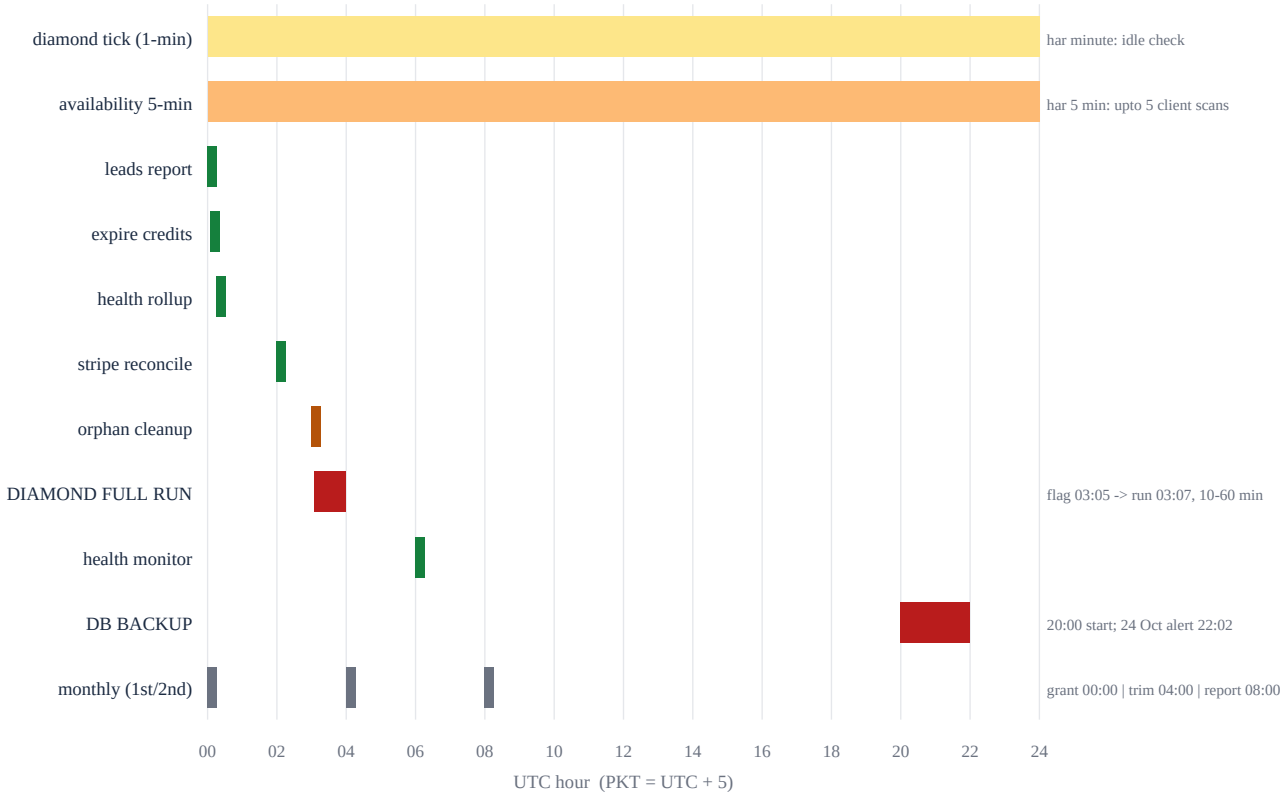
#	Job name (cron.job)	Schedule (UTC) / PKT	Runs/ day	Kya karti hai	Load	Est. duration	Repo file
15	(dynamic) marketing campaign minute tick	* * * * * sirf jab campaign sched uled/sending ho	0 nor mally	AGENTS.md ke mutabiq trigger job banata hai, idle hone par tick khud unschedule karta hai. Repo mein iski migration nahi mili - live cron.job mein check karein.	LOW	1-5 s per tick	marketing-c ampaign-tic k fn

Unschedule ho chuki (confirm karein ke live DB mein bhi nahi hain): daily-subscription-grace-enforcement (10:00 UTC, hataai gayi 2026-09-08 payment-drift migration mein), monthly-client-report (1st 13:00-18:00 UTC hourly, hataai gayi 2026-09-02).

Hidden side-effect: pg_cron har run ki ek row cron.job_run_details mein likhta hai. 1,440 + 288 + 10 = ~1,740 rows/day = ~630k rows/saal. Repo mein is table ko prune karne wali koi job nahi. Supabase khud recommend karta hai ke ise saaf kiya jaye.

3. 24-Ghante ki Timeline (UTC)

Neeche har job ka slot dikhaya gaya hai. Lambi bars bhari jobs hain. Note karein ke **03:00-03:07 UTC** par cleanup, nightly flag aur full diamond run ek saath aate hain, aur **20:00 UTC** par backup akela 1-2 ghante chalta hai. 5-min availability aur 1-min tick poore din background mein hain.



24 Oct ka alert kaise fit hota hai: 23 Oct 19:38 UTC par duplicate-merge migration chali (pool 2.15M -> ~794k, bara DELETE/merge). 24 Oct 20:00 UTC par backup shuru hua; agar us waqt pool abhi bara tha ya scores table bhari thi to OFFSET pagination 2+ ghante DB ko parhti rahi. Alert 22:02 UTC. Saath hi lab sheets ki re-upload (currency fix ke baad) har upload par pipeline tick ko full run karwa rahi thi. Dono ek saath = read/write capacity khatam. Lovable ne sirf doosra hissa dekha.

4. Bhari Jobs ka Tafseeli Jaiza

4.1 Diamond pipeline (jobs #1 + #2) - sab se zyada CPU/IO

Ek full run mein ye sab hota hai (code: score_diamond_pool + diamond_pipeline_tick):

- COUNT(*) poore pool par, phir poora pool temp table t_base mein copy (794k rows, work_mem 256 MB).
- 2 stone types x 4 cohort levels x 2 trim rounds = regression fits dynamic SQL se; har level t_base par group-by/join.
- TRUNCATE diamond_pool_scores + 794k rows INSERT; DELETE + INSERT diamond_pool_rep_changes.
- 3 materialized views REFRESH CONCURRENTLY (grade_stats, group_reps, group_reps_engine) - har ek pool ka full scan + DISTINCT ON sort. CONCURRENTLY ka matlab doubled work (nayi copy + diff).
- refresh_diamond_pool_distinct_values(): 9 alag DISTINCT scans + 1 COUNT = pool 10 baar aur parha jata hai.
- Kul milakar pool ~15-18 baar poora parha jata hai aur ~1.6M rows likhi jati hain. Statement timeout 1 ghanta is liye rakha gaya ke ye waqai lamba chal sakta hai.

Kab chalta hai: (a) har raat 03:07 UTC zabardasti, chahe pool mein koi change na ho; (b) har CSV upload ke 2 minute baad (statement trigger trg_diamond_pool_mark_dirty). Agar aap ek din mein 5 sheets upload karein to 5 full runs. **Memory se:** lab sheets dobara upload honi hain, yani aur runs aa rahe hain.

Cost share (estimate): cron-driven DB kaam ka 50-65%. Raat ko 10-60 min 100% CPU + heavy IO.

4.2 daily-jewlx-db-backup (job #4) - Lovable ne miss kiya

- Edge function list_public_tables() se HAR table leti hai - koi exclusion list nahi. Pool (794k), scores (794k), rep_changes, analytics_events, cron logs - sab.
- fetchAllRows() 1000 rows ki pages .range(from, from+999) se parhta hai. PostgREST ye OFFSET banata hai; offset 700,000 wali query Postgres ko 700k rows skip karni parti hain. 794 pages x barhti hui offset = quadratic read cost. 2.15M rows par ye 2,150 requests thi.
- Sab kuch edge function ki memory mein jama hota hai phir gzip. Supabase edge function limits (memory/wall-clock) se bare tables par ye silently fail hoti hai (tableStats = -1) lekin DB ko tab tak poora parh chuki hoti hai. backup_logs table mein ye dikh jayega.
- Faida kam, nuqsaan zyada: Supabase khud PITR/daily backups deta hai; diamond pool to waise bhi re-uploadable CSV data hai.
- Security note: cron command mein anon JWT literal hai (migration file mein commit shuda).

Cost share (estimate): 25-35%. 20:00-22:00 UTC (01:00-03:00 PKT) ka spike.

4.3 refresh_stale_diamond_availability_5min (job #3) - chupi hui job

- Repo se 2026-09-10 ko revert ho gayi thi lekin Supabase dashboard se apply ho chuki thi; is liye live DB mein ab bhi hai. Repo aur DB ka farq - koi bhi code review ise nahi dekhta.
- Condition sirf is_stale nahi: computed_at < now() - 24h bhi hai. Yani HAR confirmed client ki availability har 24 ghante dobara compute hoti hai, chahe catalogue na badla ho.
- compute_client_diamond_catalog_availability() pool par WHERE diamond_pool_row_matches(p, filter) lagata hai: plpgsql function per row = sequential scan, koi index use nahi hota. Agar client ne shapes blank chhori hon to ek aur full scan regexp ke saath.
- N confirmed clients = N full-pool scans/day, din bhar har 5 min 5-5 kar ke. Ye wo 'background hum' hai jo poore din CPU kha raha hai.

Cost share (estimate): 10-20%, lekin din bhar phaila hua.

4.4 Baqi medium jobs

• **trim_old_log_tables_monthly (#12):** analytics_events ka 90-din purana data ek DELETE mein. Mahine ka sab se bara single write; autovacuum ke liye bloat chhorta hai. Batch delete (10k rows per loop) behtar hai, ya table partitioning.

• **cleanup_orphan_ai_previews_daily (#5):** row-by-row loop with 4 EXISTS per object. Theek hai jab tak storage.objects chhota hai; 03:00 ka slot diamond run se takrata hai - 01:00 par shift karein.

• **cron.job_run_details:** ~1,740 rows/day, kabhi saaf nahi hoti. platform health monitor ise parhta hai, to table jitni bari, utna slow.

5. Cost Analysis

Supabase/Lovable compute billing instance size par hoti hai, per-query nahi. Is liye 'konsi job kitni costly' ka asal matlab hai: **kaunsi job aap ko bari instance par rehne par majboor kar rahi hai.** Lovable ke diye gaye rates:

Size	Memory	Credits/hour	~30 days	Note
Medium	4 GB	0.33	~240	Target size jab bhari jobs theek ho jayen
Large (current)	8 GB	0.62	~450	24 Oct ko upgrade kiya gaya
X-Large	16 GB	1.24	~890	Zaroorat nahi - problem CPU/IO patterns hain, memory nahi (36% used)

Estimated hissa cron-driven DB load mein (static analysis; exact % Appendix A ki SQL se nikalein):

Job	Load share	Kyun	Fix ke baad
Diamond pipeline full run (#1/#2)	50-65%	15-18 full-pool reads + 1.6M writes per run; roz + har upload	Nightly sirf tab jab pool badla ho; distinct values aur matviews ek pass; ~70% kam
DB backup (#4)	25-35%	Har table OFFSET-paged full read, 2 bare tables	Pool/scores exclude ya job band -> ~95% kam
Availability 5-min (#3)	10-20%	Per-client index-less full scans har 24h	24h re-check hatao, sirf is_stale; set-based query -> ~90% kam
Monthly trims (#12/#13)	spike 1x/month	Bara single DELETE	Batch delete
Baqi 10 jobs	< 2%	Chand rows / HTTP	Koi change nahi

Paisay ka natija: Large par rehna = ~450 credits/month. Section 6 ke fixes ke baad Medium kaafi hai = ~240 credits/month, yani **~210 credits/month ki bachat**, aur alerts bhi band. X-Large bilkul nahi: Lovable bhi yahi kehta hai aur code bhi yahi dikhata hai (memory 36% hai, problem scans hain).

5.1 Lovable ke response ka review

Lovable ne kaha	Meri raye	Status
DB ab pressure mein nahi, upgrade ki zaroorat nahi	Sahi. Memory 36%, 15/160 connections. Masla bursty jobs ka hai.	Agree
Alert ki wajah nightly diamond re-check + clean-up (24 Oct)	Aadha sahi. Pipeline runs ne hissa dala, lekin 22:02 UTC alert 20:00 backup ke 2 ghante baad aaya; backup 2.15M rows OFFSET-page kar raha tha. Lovable backup ko count hi nahi kar raha.	Partly
Diamond jobs 'ek ke baad ek' chalti hain, time limit lagao	Theek direction, lekin time limit se job adhuri reh jayegi (scores table TRUNCATE ho kar khali reh sakti hai). Behtar: kaam kam karo, limit nahi.	Partly
Chand din baad Medium par wapas jao	Sahi, lekin SIRF backup + availability + pipeline fixes ke baad. Warna 20:00 UTC spike Medium par phir alert dega.	Agree (conditional)
'Logging page visits ~1 ms sab se slow regular task'	Ye average hai; nightly batch jobs is metric mein nahi dikhte kyun ke wo 'regular' nahi. Misleading comfort.	Caution

6. Hal: Priority ke hisaab se Action Plan

Order ahem hai: pehle wo jo sab se zyada load hatayein aur sab se kam risk rakhein. Har step ek chhoti migration hai; koi widget ya pricing logic nahi badalta.

#	Action	Kaise	Load impact	Risk	Effort
1	Backup se bare tables nikaalo (ya job band karo)	database-backup mein exclusion list: diamond_inventory_pool, diamond_pool_scores, diamond_pool_rep_changes, diamond_pool_grade_stats/reps, analytics_events, cron logs. Behtar: Supabase ke built-in daily backups/PITR par rely karo aur ye job unschedule karo. Anon key ko cron command se hata kar cron_config se parho.	-25..35% (20:00 UTC spike khatam)	Low (built-in backups maujood)	1 ghanta
2	Nightly diamond run sirf tab jab pool badla ho	diamond-pipeline-nightly ko: UPDATE ... SET pending=true WHERE last_run_finished_at < last_pool_change_at (ya 7 din se purana). Pool na badle to raat ko kuch nahi chalta.	-30..40% on quiet days	Low	30 min
3	Availability sweep theek karo	refresh_stale_diamond_availability: 'computed_at < 24h' shart hatao (sirf is_stale), ya 24h -> 7 din. compute_* mein diamond_pool_row_matches ko set-based WHERE (currency + shape + carats columns, maujooda indexes) se replace karo. Repo mein migration bhi commit karo taake DB aur code match karein.	-10..20%, din bhar ka hum khatam	Low-Med (results same, sirf tez)	2-3 ghante
4	Pipeline ek pass mein	refresh_diamond_pool_distinct_values ke 9 scans ko ek GROUP BY query (ya t_base se) banao. Matviews non-concurrent refresh raat ko (koi user 03:07 UTC par nahi), concurrent sirf upload-triggered runs par. Debounce 2 -> 10 min taake multi-sheet upload ek run bane.	full run 30-50% chhota	Low	2 ghante
5	Timing phailao	cleanup_orphan 03:00 -> 01:00; diamond nightly 03:05 -> 03:30 (Stripe 02:00 se door); backup (agar rakha) 20:00 -> 04:30 UTC jab pipeline khatam ho chuki ho.	Overlap khatam	None	15 min
6	Housekeeping	cron.job_run_details prune job (7 din rakho) weekly; trim_old_log_tables ko 10k-row batches mein; daily-subscription-grace-enforcement aur monthly-client-report live DB mein confirm-delete.	Steady growth ruke	None	1 ghanta
7	Downgrade	Steps 1-4 ke 3-4 din baad Supabase Reports mein CPU/IO dekho; agar 03:00-04:00 aur 20:00 UTC ke peaks < 50% hon to Medium par jao.	~210 credits/month bachat	Low (reversible)	10 min

Kya NAHI karna: (a) X-Large par jana - masla memory ka nahi; (b) diamond_pipeline_tick par 10-min statement_timeout lagana jaisa Lovable suggest kar raha hai - TRUNCATE ke baad timeout aaya to diamond_pool_scores khali reh jayegi aur Ring OS/Quote Estimator ko stones nahi milenge; (c) 1-minute tick band karna - uploads ke baad pipeline chalni zaroori hai, wo waise bhi idle par ~2 ms leta hai.

Expected result: cron-driven DB load 70-80% kam, raat ke 2 spikes (03:07 aur 20:00 UTC) khatam, Medium instance safe, aur analytics/health monitoring same rahe.

Appendix A: Exact numbers nikaalne ki SQL (Supabase SQL Editor)

Ye queries read-only hain. Inhe chala kar aap is report ke estimates ko asal durations se replace kar sakte hain.

A1. Live cron jobs ki list (repo se compare karein - job #3 yahan dikhni chahiye, grace/monthly-report nahi):

```
select jobid, jobname, schedule, active, left(command, 90) as command
from cron.job order by jobname;
```

A2. Pichle 7 din har job ka avg/max duration aur failures (yahi 'kaunsi job kitna waqt leti hai' ka asal jawab hai):

```
select j.jobname,
       count(*)                                          as runs,
       round(avg(extract(epoch from d.end_time - d.start_time))::numeric, 2) as avg_sec,
       round(max(extract(epoch from d.end_time - d.start_time))::numeric, 2) as max_sec,
       round(sum(extract(epoch from d.end_time - d.start_time))::numeric/60, 1) as total_min_7d,
       count(*) filter (where d.status <> 'succeeded')    as failed
from cron.job_run_details d join cron.job j on j.jobid = d.jobid
where d.start_time > now() - interval '7 days'
group by j.jobname order by total_min_7d desc;
```

A3. Diamond pipeline ke har run ki step-wise timings (function khud jsonb mein save karta hai):

```
select started_at, finished_at, finished_at - started_at as duration,
       trigger_source, status, refresh_mode, pool_rows, timings
from public.diamond_scoring_runs order by started_at desc limit 20;
```

A4. Backup job kitna waqt le rahi hai aur kaunse tables fail ho rahe hain:

```
select created_at, backup_type, status, duration_ms/1000.0 as seconds,
       total_records, array_length(tables_backed_up,1) as tables, error_message
from public.backup_logs order by created_at desc limit 14;
```

A5. Availability sweep: kitne clients har din dobara compute hote hain:

```
select count(*) filter (where f.is_confirmed) as confirmed_clients,
       count(*) filter (where c.is_stale) as stale_now,
       min(c.computed_at) as oldest_cache
from public.client_diamond_catalog_filter f
left join public.client_diamond_availability_cache c on c.client_id = f.client_id;
```

A6. Table sizes (kaun si tables backup aur scans ko bhari bana rahi hain):

```
select relname, n_live_tup, pg_size_pretty(pg_total_relation_size(relid)) as size
from pg_stat_user_tables order by pg_total_relation_size(relid) desc limit 15;
```

A7. cron.job_run_details ka size aur row count:

```
select count(*), pg_size_pretty(pg_total_relation_size('cron.job_run_details'))
from cron.job_run_details;
```

Appendix B: Hawalajaat (repo files)

Topic	File
Diamond engine, tick, nightly flag	supabase/migrations/20260925090000_diamond_outlier_engine.sql (lines 1330-1350)
Latest tick body (distinct-values cache)	supabase/migrations/20261025120000_diamond_pool_distinct_values_cache.sql
Latest score_diamond_pool + matviews	supabase/migrations/20261022120000_ring_os_catalog_currency_filter.sql
Backup cron + edge fn	supabase/migrations/20260421135203_*.sql ; supabase/functions/database-backup/index.ts
Availability compute / refresh	supabase/migrations/20260730195243_*.sql ; 20260910124634_*.sql ; deleted 20260910130515 (git show ff79772d)
Cleanup / trim / archive	supabase/migrations/20260422182534_*.sql ; 20260423000001_schedule-cleanup-crons.sql
Health monitoring crons	supabase/migrations/20260918180000_platform_health_monitoring.sql
Leads report / Stripe reconciliation / credits	20260921180000, 20260418160000, 20260416165255, 20260417100000